

访谈笔记

访谈笔记：陈天奇 — 机器学习系统，长期主义与初心

基于公开课程资料整理

2025-09-12



视频作者/频道：WhynotTV

发布日期：2025-09-12

视频时长：160 分钟

视频链接：<https://www.bilibili.com/video/BV1s6pgzLE3y/>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 嘉宾介绍与访谈背景	3
2 成长经历：从县城到 ACM 班	3
2.1 童年与计算机的相遇	3
2.2 自学编程与信息竞赛	3
2.3 交大 ACM 班的历练	4
2.4 本章小结	5
3 早期科研：失败的价值	5
3.1 APEX 实验室与深度学习初体验	5
3.2 KDD Cup 的转机	5
3.3 香港实习：研究视野的打开	6
3.4 本章小结	6
4 XGBoost：从假说挑战到行业标准	6
4.1 诞生背景	6
4.2 算法与系统的垂直整合	7
4.3 树模型 vs 神经网络的反思	7
4.4 导师 Carlos Guestrin 的影响	7
4.5 本章小结	8
5 MXNet：深度学习框架的百花齐放	8
5.1 从 CXXNet 到 MXNet	8
5.2 核心设计理念	8
5.3 与 TensorFlow 的竞争	9
5.4 李沐的中流砥柱作用	9
5.5 本章小结	9
6 TVM：机器学习编译的开拓	10
6.1 从问题出发的动机	10
6.2 GPU/TPU/NPU 的区别与融合	10
6.3 “荒岛建城”的开发过程	10
6.4 Jeff Dean 的坐标系与高风险选题	11
6.5 博士后期的勇气与 Recomputation 的故事	11
6.6 TVM 的演化与生态位	12
6.7 本章小结	12
7 MLC LLM：大模型时代的端侧部署	13
7.1 从 TVM 到 MLC LLM	13
7.2 端侧 AI 的未来	13
7.3 本章小结	13

8 机器学习系统的 20 年演化	14
8.1 三大浪潮	14
8.2 系统对算法的使能作用	14
8.3 学术界的机遇与挑战	15
8.4 MLSYS 的未来方向	15
8.5 本章小结	16
9 OctoML: 从开源到创业	16
9.1 创业动机	16
9.2 商业模式的转变	16
9.3 创业最大的 Lesson	17
9.4 本章小结	17
10 开源的信仰与实践	18
10.1 为什么坚持开源	18
10.2 构建开源社区的关键	18
10.3 本章小结	18
11 教职生涯：在一线写代码的教授	18
11.1 选择 CMU	18
11.2 坚持一线写代码	19
11.3 捕鱼的比喻：积淀与时机	19
11.4 Advising style	19
11.5 教职生涯本章小结	19
12 人生哲学：长期主义、初心与勇气	20
12.1 长期主义的数学基础	20
12.2 勇气来自过去的自己	20
12.3 保持初心是最难的事	20
12.4 关于成功与幸福	21
12.5 人生哲学本章小结	21
13 总结与延伸	21
13.1 一条清晰的技术主线	21
13.2 对年轻研究者的启示	22
13.3 对 AI 未来的展望	22
13.4 结语	23
13.5 拓展阅读	23

1 嘉宾介绍与访谈背景

本期 WhynotTV Podcast (第三期) 的嘉宾是陈天奇 (Tianqi Chen), CMU 机器学习系助理教授, OctoML 联合创始人 (2024 年底被 NVIDIA 收购)。过去近 20 年, 陈天奇围绕 **“让机器学习更轻、更快、更易部署”** 这条主线, 先后主导了 XGBoost、MXNet、TVM、MLC LLM 等广受欢迎的开源项目。其中 XGBoost 论文引用数接近 7 万次, 至今仍是 tabular data 领域最常用的工具。

2019 年他在知乎发表《机器学习科研这十年》, 收获上万点赞。六年后的 2025 年, 他在机器学习这条路上已经走了近 20 年。主持人 (WhynotTV) 指出: “对我而言, 陈天奇不仅仅是 AI 领域一个响亮的名字, 他还影响了我对机器学习和科研最早的认知。”

本次访谈长达 160 分钟, 从他的童年经历出发, 经过本科 ACM 班、研究生 APEX 实验室、UW 博士期间的四大项目 (XGBoost、MXNet、TVM、Google Brain 实习的 Net2Net), 到 CMU 教职和 OctoML 创业, 最后上升到长期主义、初心和勇气等人生哲学。

陈天奇的技术主线

从 XGBoost (2014) 到 MXNet (2015) 到 TVM (2017) 再到 MLC LLM (2023), 陈天奇的每一个项目都围绕同一个核心问题: **如何用更少的工程资源、更自动化的方式, 让机器学习系统在各种硬件上高效运行**。这条主线贯穿了大数据时代、深度学习时代和大模型时代。他不是一个绑定在某种方法上的研究者, 而是一个追踪问题的人——方法可以变 (从树模型到深度学习框架到编译器), 但问题始终是“让 ML 系统更好”。

2 成长经历：从县城到 ACM 班

2.1 童年与计算机的相遇

陈天奇的中学在浙江省丽水市松阳县的松阳二中——他母亲是那里的高中老师, “当时觉得自己在自己地方上学也挺好的”。主持人问他童年对什么最感兴趣, 他说最有趣的经历还是怎么开始接触计算机。

学习计算机的最初 motivation 是**想自己做电子游戏**——“学计算机的一个 motivation 就是能够自己创造游戏。当然这个 motivation 一直没有实现。”但正是这个对“创造”的渴望, 引领他进入了编程世界, 并一直延续到后来的科研中——“做机器学习也是这样的, 创造智能。创造东西是一个原始的 motivation。”

2.2 自学编程与信息竞赛

初中时, 他出于对做网页的向往找到了计算机启蒙老师何老师, 得到一本 C 语言入门书。在没有竞赛教练、没有 GitHub 的年代, 他通过网上的 Online Judge (OJ) 和论坛 (如“大龙树”) 自学编程与信息竞赛。

自学者的“野路子”优势

当时主流竞赛语言是 Pascal, 但陈天奇因为入门时只学了 C, 就一直用 C 打比赛。这种“野路子”反而培养了他不按套路出牌的习惯。高二时他出于兴趣, 花了一个暑假从零写了一个 Pascal 到 C 的 transpiler (编译器雏形)。这段经历让他“不怕干事情”, 奠定了日后从 first principle 出发解决问题的风格。

高二参加 NOIP 初赛未过，旁听决赛拿了二等奖——当时看到结果“还蛮伤心的”。但第二年高三时拿到一等奖，而且高考完全没有落下，正常考入上海交通大学电院。主持人追问：“你当时在县城是不是一直全年级第一？”陈天奇笑答：“差不多算是吧。”

值得注意的是，学习编程的最初动机其实是想自己做电子游戏——这个 motivation 一直没有实现，但引领他进入了一个更广阔的世界。

关于“出生牛犊不怕虎”

“出生牛犊不怕虎”是陈天奇在访谈中反复使用的词。从高中自学竞赛、到写 transpiler、到后来做 MXNet 挑战 TensorFlow、做 TVM 进入编译器领域，每一次他都是在“不知道会碰到什么问题”的情况下直接上手。这种心态的形成与他在小县城自学、没有权威告诉他“这个很难”有关——没有见过天花板，反而不怕天花板。

2.3 交大 ACM 班的历练

2006 年，陈天奇通过高考考入上海交大电院，然后在宣讲会上认识了俞勇老师，精心准备材料后加入了 ACM 班。他回忆说，于（俞）老师后来跟他说“你是准备材料最仔细的一个学生”——这非常像后来准备 PhD 申请的 personal statement，“但当时并没有意识到这一点”。他作为一个“名不见经传的地方来的高考生”，能加入 ACM 班是“非常幸运的”。

ACM 班对他的影响体现在几个维度：

1. **表达能力**：ACM 班的“教子讲坛”要求每个学生每学期选一个 topic 上台讲解，这培养了他在读博时直接 volunteer 做组会报告的自信。
2. **“低调做人、高调做事”**：俞勇老师反复强调的价值观。“做一件事情要做到极致”成为他日后做 XGBoost 的核心驱动力。
3. **编译器大作业**：ACM 班的编译原理课程设计要求从零搭建编译器，由第一届学长自发借鉴北美高校经验设计，一代一代传承。陈天奇认为这个训练“哪怕放到现在北美高校的标准来看都是一流的”。
4. **体系结构大作业**：他在 ACM 班还挑战了在自己写的 CPU 上运行自己写的编译器。

系统功底的积累

ACM 班的编译器 + CPU 大作业经历，为陈天奇日后设计 TVM 编译器和 VTA 加速器打下了坚实基础。他回忆 TVM 开发时说：“因为有了原来那段经历，让我觉得不是很慌。”

“低调做人、高调做事”的传承

俞勇老师在 ACM 班反复强调的“低调做人、高调做事”深刻影响了陈天奇。他后来在职业生涯中接触各种成功人士后发现，真正能成事的人有两个特质：一是 ambitious，有锐气去挑战现状（challenge the status quo）；二是 humble，知道成功的因素“几乎不全部来自于你自己”，需要团队合作。这两者的结合——谦逊中保持锐气——成为他一贯的处事风格。

2.4 本章小结

陈天奇的早期成长呈现出一个反复出现的模式：**在资源匮乏的环境中自学、不怕失败、从 first principle 出发解决问题**。从县城自学 OI，到高中写 transpiler，到 ACM 班写编译器跑在自己的 CPU 上，每一段经历都在为日后的大项目积累“不怕干事情”的勇气和系统设计的直觉。

3 早期科研：失败的价值

3.1 APEX 实验室与深度学习初体验

本科期间，陈天奇加入交大 APEX 实验室，跟随学长戴文渊（后创办第四范式）入门机器学习，读 Tom Mitchell 的经典教材。选择机器学习的动机很单纯：“机器可以学习听起来很酷”。Tom Mitchell 正是 CMU 机器学习系的第一任系主任，陈天奇后来到 CMU 任教时感慨：“很多机器学习教授都是看着那本书长大的。”

大三时薛老师推荐他们关注深度学习，他和学长林源用受限玻尔兹曼机（RBM）做 ImageNet 分类。为了跑 GPU，他们买了 GTX 380 显卡，发现机箱电源功率不够，于是把额外电源放在机箱外面，搭建了实验室第一台 GPU 服务器。这是 2009-2010 年的事，距离 AlexNet 还有两年。当时 Andrew Ng 组已经有论文讨论用 GPU 加速深度学习，Alex Krizhevsky 的 CUDA ConvNet 工具也已经可以在网上找到，但 GPU 深度学习还远未成为共识。

陈天奇的毕业设计就是做深度学习，而且是“从零用 C++ 和 CUDA C 写 backpropagation”——当时虽然 Theano 和 Caffe 的前身已经存在，但他选择 from scratch 实现，“因为觉得很多东西从头写还是可以知道控制你的模型建模是怎样的”。搭建一个模型要花六个月时间写代码，然后再跑实验。

正确的问题 + 错误的方法

陈天奇从本科毕设到研二，花了近两年时间做“深度学习 on ImageNet”，用的是卷积 RBM 方法。结果始终无法超越 SVM baseline。后来回头看，深度学习方向是对的，但 RBM 路径本身不对——最终 work 的是 supervised learning（如 AlexNet 的卷积网络）。他总结：**不能同时锁死要解决的问题和要使用的方法**。挑了正确的问题后，方法要灵活调整。

3.2 KDD Cup 的转机

研究生最后一年，团队参加 KDD Cup 推荐系统比赛。当时他们知道有一篇论文叫“Restricted Boltzmann Machines for Collaborative Filtering”（Ruslan Salakhutdinov 等人的工作），于是尝试把 RBM 用在推荐系统上。但很快他们意识到矩阵分解更有效，果断转换方法后取得了好成绩。陈天奇后来做的开源项目 SVD Feature 就是这段工作的产物，Danny（Carlos Guestrin 的 postdoc）通过这个项目发现了陈天奇，在博客上做了简单访谈，并把他推荐给了 Carlos 和 Alex Smola。这个契机直接促成了他获得 UW 的 PhD offer。

他同时拿到了 CMU 的 offer，但因为想跟 Carlos 做研究而选择了 UW。

开源项目的“隐性回报”

SVD Feature 作为一个“现在已经没有人知道了”的早期项目，却成为陈天奇进入北美学术圈的敲门砖。开源项目的价值不仅在于代码本身，更在于它让你的工作被潜在合作者发现。这个模式在他后来的每一个项目中都反复出现：XGBoost 的社区效应、TVM 吸引硬件厂商关注、MLC LLM 在端侧社区的影响力。

3.3 香港实习：研究视野的打开

李航老师来 APEX 做报告后，陈天奇发现“原来世界上还有这些其他问题”，而且凭借之前的方法积累，很多问题他都能想到解决方案。在香港跟随李航老师和杨强老师实习期间，他开始明白：科研不仅是解决问题，更重要的是知道有哪些问题可以解决。

两种科研风格

陈天奇意识到科研有两种 style：(1) 从方法出发，成为某个方法的专家，把方法用在各种问题上；(2) 从问题出发，对某类问题感兴趣，用不同方法解决它。他属于后者——**问题驱动型研究者**。这也是为什么他能从 XGBoost 跨到 MXNet、再跨到 TVM，因为他追踪的不是某个方法，而是“让 ML 系统更好”这个问题。

3.4 本章小结

早期科研的“失败”经历带来了三个关键收获：(1) 学会了接受失败，从而更敢放手做事；(2) 认识到不能同时固定问题和方法；(3) 通过大量基础训练积累了“内功”，为后来快速进入新领域打下基础。正如他所说：“如果当时做的特别成功，反而有可能形成路径依赖。”

4 XGBoost：从假说挑战到行业标准

4.1 诞生背景

2013 年 AlexNet 横空出世后，陈天奇和导师 Carlos Guestrin 讨论：深度学习成功的原因到底是神经网络本身，还是大 hypothesis space + 大数据？如果后者是关键，那树模型也有 arbitrary grow 的能力（不断增加树的数量和深度），或许也能在大数据上取得突破。

他们决定用树模型来挑战这个假说。陈天奇基于 KDD Cup 的经验，坚持选择 tree-based method（而非导师偏好的 kernel method）。他认为树模型有一个很强的特性：可以随着数据量 arbitrary grow depth，而 kernel method 虽然也是 nonparametric，但在 tabular data 上没有同样的优势。Carlos 给了他充分的自由度——“我非常感谢我的导师，他给了我非常大的自由度。”

XGBoost 的名字是 eXtreme Gradient Boosting，“extreme”体现了做到极致的追求。

XGBoost 成功的三个关键

1. **极致性能**：在发布时是世界上最快的 gradient boosting 实现。陈天奇把每一个优化都做到极致，这源自 ACM 班“做一件事情要做到极致”的传统。
2. **开源社区**：XGBoost 的成功很大程度来自社区贡献。例如一位法国律师贡献了 R 语言可视化包，用于检测税务欺诈。社区力量让项目远超个人能力范围。
3. **专注**：只做一个算法，把这一个算法做好做到极致。相比之前的“大而全”项目 SVD Feature，XGBoost 的专注策略更适合小团队。

4.2 算法与系统的垂直整合

XGBoost 内置了对 missing value 的自动处理：如果某个特征值缺失，模型会根据历史 pattern 自动决定如何分类，不需要用户预先做数据清洗。这个功能“非常非常受欢迎”，因为实际数据中缺失值无处不在。传统做法是用中位数等方式填充，但 XGBoost 让模型自己学习最优的处理方式。

陈天奇在做 XGBoost 时第一次意识到：**做机器学习系统，只做算法或只做系统都不够，必须垂直整合**。这个认知贯穿了他此后所有项目。他指出，XGBoost 的成功不仅因为系统层面跑得快（多线程、out-of-core learning、分布式计算），也因为算法层面的细节优化（正则化、sparsity-aware split finding、missing value handling）。**只有同时理解算法的需求和系统的约束，才能做出真正好用的工具。**

XGBoost 论文的写作哲学

XGBoost 的论文直到 PhD 第三年才写。Carlos 的要求不是追求中稿，而是“写一篇让读者可以学到东西的文章”。同年 KDD 上 Carlos 实验室还有另一篇 LIME (Explainable ML)，由 Marco Ribeiro 完成，开创了可解释机器学习这个方向。Carlos 的研究 style 是鼓励学生做能开创一个领域的工作，而不是在已有方向上做增量改进。

4.3 树模型 vs 神经网络的反思

十多年后回顾，他认为当时的判断“partly right, partly wrong”：

- **正确的部分**：模型 capacity 不应受限制。树模型可以 arbitrary grow depth，这个特性很强。在 tabular data 和 time series 上，树模型至今仍是 go-to tool。
- **没有预料到的**：神经网络通过一种“更暴力”的方式实现了 capacity 扩展——不是逐渐 grow parameter space，而是一开始就把 parameter space 设得非常大。
- **神经网络的组合特性**：transformer block、linear block 等模块可以相互组合，使得表达能力通过组合更 compact 地表示。

4.4 导师 Carlos Guestrin 的影响

Carlos 对学生要求极高：第一次见面就说“You should write the best paper in the world”。他的实验室有“no beamer, only PowerPoint”的规定，强调用视觉化方式让读者快速理解研究内容。XGBoost 论文直到博士第三年才写，Carlos 的要求是“写一篇让读者可以学到东西的文章”，而不是优化中稿率。

4.5 本章小结

XGBoost 的成功不是偶然：极致的性能、活跃的社区、专注的策略，加上算法与系统的垂直整合，使它成为引用近 7 万次的行业标准。但陈天奇当时并没有预料到这个成功——纯粹是“做了一个我们觉得还行的东西，然后开源了”。

5 MXNet：深度学习框架的百花齐放

5.1 从 CXXNet 到 MXNet

AlexNet 的成功重新点燃了陈天奇对深度学习的热情。他和许斌（现 NVIDIA）开发了 CXXNet，使用 C++ 模板编程（expression template）技术生成内联 GPU kernel。这个技术的核心思想是：你在写 update rule（比如做乘法、减法等运算）时，它可以通过 C++ 模板把这些操作压缩成一个 kernel，生成内联的 GPU 代码。CXXNet 的性能不错，还被用来打 Kaggle 深度学习相关的比赛。

与此同时，深度学习框架领域正处于“百花齐放”的阶段：

- **张真 + 敏捷**（MSR/NYU）：开发了 Minerva，Python first 的设计理念在当时非常先进
- **李沐**（CMU）：做 Parameter Server（参数服务器），需要分布式作为 first class citizen
- **张池元**（MIT）：用 Julia 写了一个叫 Moka 的框架
- **林敏**（NUS）：做了 Purine 框架
- **杨青**（UC Berkeley）：Caffe 已经 release，是当时的主流框架，但核心是 C++

通过 NeurIPS 等学术会议的契机，这些分散在 UW、CMU、MIT、NUS、NYU 的 PhD 学生聚在一起，决定合力做一个统一的框架——这就是 MXNet 的起源。

MXNet 的名字由来

MXNet 是 Minerva 和 CXXNet 的混合（Mix），同时也代表“mix programming style”——混合声明式计算图（declarative，先声明计算图再执行）和命令式编程（imperative，写了就直接算）。当时这两种范式存在明显的 trade-off：声明式对系统优化有利（可以做全局优化），命令式对开发者更友好（Debug 简单）。MXNet 的选择是两者都支持。后来 PyTorch 选择了纯命令式（eager mode），TensorFlow 1.x 选择了纯声明式（后来 2.0 也转向了 eager mode），MXNet 一开始就提供了混合模式。

陈天奇还提到，UW 的同组同学中有 Joe Redmon（YOLO 的创始人），“他在博士之前就做了 YOLO”。AlexNet 出来后，组里很多人都开始探索深度学习方向。

5.2 核心理念

MXNet 的三大创新：

1. **Python First**：在 Caffe 以 C++ 为核心的年代，MXNet 选择以 Python 为一等公民。
2. **自动调度**：根据计算依赖关系自动实现多卡并行，通信和计算可以 overlap。
3. **自动求导**：从 Caffe 的手写 backpropagation 进化到基于计算图的自动求导。

5.3 与 TensorFlow 的竞争

陈天奇暑假在 Google Brain 实习时，发现 TensorFlow 即将发布。他是 TensorFlow 的 dogfood 用户之一，同时也向 Brain 团队分享了 MXNet 的设计经验。但团队并没有退缩：“出生牛犊不怕虎，我们也可以 compete 一下。”甚至“很有激情地继续往前推动”。

时间线上，MXNet 的立项早于 TensorFlow 的发布。大家在前一年的 NeurIPS 上聚在一起决定往前走，TensorFlow 在 2015 年底 release，MXNet 的正式发布约在同期。最终 MXNet 只投了一篇 workshop paper，作者按字母序排列——“纯粹就是觉得一起做一个项目挺好的”。

PhD 学生的“纯粹”合作

主持人问：这么多 PhD 聚在一起，没有人管事，怎么能 work out？陈天奇回答：“根本没有人可以说了算。因为大家纯粹是想要一起做项目。”这种基于共同兴趣、没有层级关系的合作模式，在学术开源社区中反而有其优势——每个人都是出于技术热情而非 KPI 在贡献。合作中“就技术讲技术其实很简单，直接讨论方案然后往前走就可以了”。

MXNet 淡出历史舞台的教训

MXNet 最终被 PyTorch 超越，陈天奇总结了两个关键教训：

1. **用户体验优先于性能**：MXNet 在多卡并行上长期是最优秀的框架（NVIDIA 跑 MLPerf 都用它），但在用户体验上不如 PyTorch。“在选择用户体验和性能上面，我们觉得都要，但其实应该先选用户体验。”
2. **工程人力不足**：为支持每一代 GPU 的迭代，需要大量工程投入，PhD 学生团队难以持续。这直接促使他转向 TVM——**用编译技术来减少工程开销**。

5.4 李沐的中流砥柱作用

陈天奇特别提到李沐是 MXNet 项目中做了“非常大的努力和牺牲”的人。“其实最后李沐是在这里面推动最大的。”李沐毕业后去了 Amazon，推动 MXNet 成为 AWS 官方框架，并开发了 GluonCV 和 GluonNLP。陈天奇指出 GluonNLP “in some sense 有点像 Hugging Face”——提供各种预训练模型的统一接口。但最终这些生态层面的努力没能改变 PyTorch 社区已经壮大的格局。

一个不太为人知的事实是：NVIDIA 在非常长的一段时间里都在 contribute MXNet、使用 MXNet。原因是在所有开源框架中，MXNet 的多卡并行性能一直是最优秀的。NVIDIA 在做 MLPerf 跑分提交时，“一定要拿到最好分数的时候，一直在用 MXNet”——直到 ResNet 不再是 benchmark 的主角之后才逐渐淡出。

陈天奇还提到图森未来的王乃岩和侯晓迪“有很长一段时间一直在帮忙贡献和支持 MXNet”。这个项目的成功和最终淡出，都离不开社区中很多人的贡献。

5.5 本章小结

MXNet 的经历教会陈天奇两件事：（1）用户体验比技术指标更重要；（2）工程人力的瓶颈需要通过更根本的技术手段（编译器）来解决，而不是简单地堆人。这两个教训直接催生了 TVM。

6 TVM：机器学习编译的开拓

6.1 从问题出发的动机

从 CXXNet 到 MXNet，陈天奇发现一个反复出现的瓶颈：每一代 GPU 和新硬件都需要大量手写 kernel，支持不同数据 layout（如 NHWC vs NCHW）、不同输入尺寸、不同硬件形态，排列组合的空间巨大。手写 kernel“根本干不过来”。

TVM 要解决的核心问题

能不能用编译技术自动生成这些 kernel？能不能让更少的人通过正确的技术，快速搭建面向新硬件的机器学习系统？这就是 TVM 的出发点——**让机器学习系统的工程复杂度不再随硬件种类线性增长**。

6.2 GPU/TPU/NPU 的区别与融合

陈天奇在访谈中科普了三种计算芯片的区别：

- **GPU**：源自图形编程，核心特性是 SIMT (Single Instruction Multiple Thread)，用大量线程并行编程。随着 CUDA 的发展，已经和图形编程“打不了关系”。
- **TPU/NPU**：拥有更复杂的指令集，一条指令可以完成大块矩阵运算或内存读写。一般没有线程概念，因为单条指令本身就非常“大”。
- **融合趋势**：GPU 引入了 Tensor Core（原本是 NPU 才有的），NPU 也开始引入线程概念以接近 CUDA 生态。“现在这个界限已经越来越模糊了。”

这种融合趋势正是 TVM 持续存在价值的体现——每一代硬件的编程模型都在变化，需要编译技术来桥接差异。

6.3 “荒岛建城”的开发过程

TVM 的灵感部分来自 Halide (MIT 的 Jonathan Ragan-Kelley 等人的工作，在图像处理领域用编译技术优化 pipeline)，但机器学习编译在当时是一片空白。陈天奇形容这段经历是“在荒岛里面建城”——没有现成的路径可以参考。

从 2016 年 4 月 GTC 回来后开始构思，到 2017 年 3 月第一个可运行版本，花了约 11 个月，基本都在写代码。他回忆这个过程是“不断迭代”的：“虽然不懂也不怕，出生牛犊不怕虎。”

在做 TVM 的同时，他和体系结构方向的合作者 Thierry Moreau 一起，用 FPGA 搭建了一个可编程的 NPU 加速器。他们最初的想法更“wild”——直接把神经网络 bake 到 FPGA 的 wire 里（后来有人做了 HLS 相关的工作）。最终他们决定先做一个类似 TPU 的 NPU，但设计理念与当时的主流完全不同。

ACM 班大作业经历的回报

陈天奇在回忆 TVM 开发时多次提到 ACM 班的经历。“我们当时在 ACM 班有一个体系结构大作业，我挑战了在自己写的 CPU 上运行自己写的编译器。TVM 非常像那段经历——也是从头搭一个编译器 for 一个新的硬件。因为有了原来那段经历之后，让我觉得不是很慌。”这说明本科阶段看似“课外活动”的挑战性项目，可能在若干年后以意想不到的方式产生回报。

陈天奇特别指出，当时他“不是做体系结构出身”，完全是 from first principle 在想怎么做。主持人评论说：“你一直都是这样——高中竞赛出生牛犊不怕虎，ACM 班写编译器出生牛犊不怕虎，做 XGBoost 出生牛犊不怕虎，做 TVM 还是出生牛犊不怕虎。这是不是你最大的优势？”陈天奇回应：“我觉得挑问题和怎么做很重要。做平庸的问题和重要的问题花的时间几乎是一样的，所以一定要挑正确的问题。”

同时，他和体系结构方向的合作者一起，用 FPGA 搭建了一个可编程的 NPU 加速器（VTA）。这个 NPU 的设计思路非常有前瞻性：

VTA 加速器的创新设计

大部分早期 NPU 是完全 hardwired（不可编程，只有一个固定功能）。但 VTA 采用了**指令级可编程**设计，可以用同一套微指令实现多种算子。为了解决内存读写和计算的 overlap 问题，他们借鉴了超线程（hyper-threading）概念：在逻辑上将 NPU 分为两个线程，一个做内存操作，一个做计算，通过硬件 signal 机制同步。这个设计“放在今天都还挺有趣”，在 NVIDIA 最新的异步编程中能看到类似影子。

6.4 Jeff Dean 的坐标系与高风险选题

在 Google Brain 实习期间，Jeff Dean 把陈天奇拉到会议室，画了一个坐标系：一轴是 risk，一轴是 impact。在不同的项目选项中，陈天奇选了 risk 最大、impact 也最大的课题——Net2Net。这个项目的的问题是：已经有一个训练好的小模型，如何快速 bootstrap 一个大模型，使其容量可以增大？

陈天奇选择这个课题的原因是他一直有一个 vision：智能需要能演化，模型的 capacity 需要变大。一个神经网络如果一开始很小，到一定程度就会不行。从 lifelong learning 的角度，怎样解决这个问题？他指出：“其实现在你会发现这个问题还是同样存在的——这也是为什么大家开始会有通过小的模型 scaling up 的方向。”

Jeff Dean 当时是他在 Brain 的 mentor，而且“非常 open minded”。陈天奇当时算是 Jeff Dean 的第一个 intern——“他的 GAN 可能还没有发布或者刚发布”的阶段。

他的选择标准是：“**时间是有限的，所以一定要做想要做的事情。它不一定是 impact 的事情，但一定要是想要做的事。**”

面对“你会建议年轻 researcher 选高风险高收益课题吗”的问题，他的回答很务实：不同人可能不一样。但因为他已经经历过很多失败，“再失败一次也没有那么差”。“Almost always 我们会 end up 选一个比较高风险高回报的方向。”

6.5 博士后期的勇气与 Recomputation 的故事

在做 TVM 之前，陈天奇曾想到一个“用低于线性复杂度训练更深模型”的方法（通过 recomputation 节省内存），但 Carlos 说这个工作可能“发了就是个 NeurIPS poster”，不够深，要做到 best paper level——应该设计一个更自动化的方法来选择 recomputation 策略。因为精力有限，他没有继续这个方向，后来有 follow-up work 做了相关工作。

Carlos 的要求不是否定这个方向的价值，而是说：“**你需要挑你做的东西，并且一旦决定要做，就要做得深了。**”

李沐在知乎评论中透露：陈天奇 PhD 第三/四年跟他说要做 TVM，李沐提醒他风险太高——PhD 快结束了做一个新的大坑。陈天奇的回答是：“不管未来去工作也好、创业也好、找教职也好，

最重要的还是要做当时自己觉得最重要的事。”李沐后来评价：“现在看起来这是唯一正确的做事方式。”

关于做选择的底层逻辑

陈天奇做选择时会问自己一个问题：“如果选了这条路并且失败了，我还会不会后悔？”如果答案是不后悔，就选它。这不是理性的投入产出分析，而是确保自己追随内心。“很多时候你后悔的不是有没有获得成功，而是你是不是走了理想中想要走的那条路。”

6.6 TVM 的演化与生态位

TVM 从诞生到现在已经经历了多轮重构，从最初的图编译到 Relax（为 LLM 重新设计的图编译器）。陈天奇强调 TVM 的设计哲学是“可演化的”——不断 reinvent 自己以适应新的需求。他指出：“你去看现在的 TVM 和它刚开始时候的形态，我们几乎已经经过了几个版本的迭代。”

面对 Torch Compile、XLA、Triton 等竞争者，他认为编译技术的需求不但没有减小，反而在增大：

- **硬件 specialized**：每一代芯片往前进的方式都不同，同一个 vendor 的不同代产品编程模式都有变化
- **模型与硬件的“缠绕”**：为达到最优效率，模型设计越来越和硬件特性耦合——DeepSeek 针对特定硬件环境做设计就是典型例子
- **Hardware Lottery**：硬件限制了模型发展的可能性，需要编译技术来打破这个束缚
- **AI Agent 的兴起**：Agent 系统和 DSL 的出现为编译技术带来了新的应用场景

TVM 目前的发展方向有两个：一是更好地与 PyTorch 等生态整合（将通用的函数调用规范单独抽象出来，方便跨框架调用）；二是在垂直领域做突破（如将 LLM 编译部署到网页和车载设备）。他的终极目标是：“像写神经网络一样简单地做机器学习工程开发”——目前这还远未实现。

Reinvent yourself vs Patch 现有系统

在 OctoML 内部，陈天奇也面临过 reinvent vs patch 的抉择。为了支持 LLM，他们推翻了 TVM 最初的图编译部分，重新做了 Relax。这个决定“不会被所有人认可”，因为已有技术短期可以解决当前问题。但他坚持认为：“If you don't reinvent yourself, there will be a time where you bite the technical debt.”长期来看，技术债务的积累比重构的短期成本更可怕。

6.7 本章小结

TVM 是陈天奇“从问题出发”研究风格的集大成之作。它源于 MXNet 开发中发现的工程瓶颈，灵感来自编译器领域，设计中融合了硬件协同设计的思想。虽然面临激烈竞争，但编译技术在 ML 系统中的核心地位已成共识。

7 MLC LLM: 大模型时代的端侧部署

7.1 从 TVM 到 MLC LLM

MLC LLM 是 TVM 的垂直应用——一个跨平台的 LLM 推理引擎，能将大语言模型编译部署到 iPhone、Android、AMD 显卡、MacBook 等各种设备上。它使用机器学习编译技术，因为团队“非常明确地知道我们并没有足够多的能力去手工铺开各个平台的支持”。

7.2 端侧 AI 的未来

陈天奇将当前的 AI 格局类比为大型机时代：“大家都用 mainframe，觉得所有东西都可以 centralize。但后来有了 personal computer。Today 大家还是习惯用自己 computer 上面的 software，虽然 Chromebook 这种 vision 还不错，但大家更多还是会有 personal computer。”端侧部署的核心驱动力包括：

1. **隐私 (Privacy)**：虽然很多人说“不 care”（主持人自嘲说“登上我的 ChatGPT 问它我所有的 social security number，估计全都知道”），但隐私需求客观存在
2. **实时性**：主持人从机器人领域的视角指出，机器人需要在毫秒级做出决策——“如果要走路、要抓一个杯子，没有时间站在那里 reason 10 秒”。NVIDIA 最近在做 Thor 芯片整合部署到机器人上，正是这个需求的体现
3. **成本**：当 pre-training 开始“start a bit”（模型能力增长趋缓），大家会从追求“next level of intelligence”转向关注 application 和 cost
4. **专用化**：local model 在特定领域达到“够用”水平后，格局就会改变——“不是要在通用领域上完全匹配最大的模型，而是在一些专用领域如果能够够用的地步，I think the picture can change”

陈天奇本人也表示：如果有一个本地 AI 能力几乎和 GPT 相当，“I would opt to try that way”。关键变量是端侧模型能力到底能有多强。

从 Mainframe 到 Personal Computer 的历史类比

这个类比非常有启发性。计算机的历史经历了从集中式大型机到分布式个人计算机的演变，AI 可能重复这个轨迹。但这不是简单的重复——云端和端侧会根据场景各有分工，就像今天的 web 应用和本地应用共存一样。决定格局的核心变量是本地模型的能力边界，以及用户对成本、隐私、实时性的权衡。

7.3 本章小结

MLC LLM 体现了陈天奇一贯的理念：通过编译技术降低工程门槛，让更小的团队也能在各种硬件上部署 AI。端侧 AI 的未来取决于模型能力和成本的平衡，但长期趋势很可能是“百花齐放”的分布式格局。

8 机器学习系统的 20 年演化

8.1 三大浪潮

陈天奇将过去 20 年的 ML 系统演化分为三个阶段，每个阶段的系统创新都直接促成了算法的突破：

第一阶段：大数据时代（2008–2012）

标志性事件是 Netflix Challenge 推荐系统比赛，以及今日头条等公司通过推荐系统起家。这个时代的突破来自于数据量从几百个数据点跃升到上亿级别。代表系统包括 Apache Spark、XGBoost，以及李沐和戴文渊在百度做的大规模线性回归（非开源系统）。

第二阶段：深度学习 Breakthrough（2012–2017）

AlexNet 的成功不仅是算法的胜利，更是系统工程的胜利：Alex Krizhevsky 用两块 GPU 训练了一个星期，在当时实验通常只跑几分钟的背景下堪称壮举。Caffe 的出现“非常大幅度地推动了 CV 领域 adopt 深度学习”。随后 MXNet、TensorFlow、PyTorch 的出现，让搭建模型从“6 个月写 2 万行 CUDA”变成“几分钟写几行 Python”。

第三阶段：大模型时代（2020–至今）

Transformer 统治了模型架构，注意力转向 scaling law、分布式训练、高效推理。新的系统挑战包括：万卡级别的训练集群、模型并行策略、端侧部署、以及结构化生成（如 XGrammar）。

阶段	时间	标志性事件	代表系统
大数据时代	2008–2012	Netflix Challenge	Spark, XGBoost
深度学习	2012–2017	AlexNet, ImageNet	Caffe, MXNet, PyTorch
大模型时代	2020–至今	GPT, Transformer	vLLM, Triton, MLC LLM

表 1: 机器学习系统的三大演化阶段

8.2 系统对算法的使能作用

陈天奇强调，每一次算法突破的背后都有系统层面的革新：

- AlexNet 之所以能成功，是因为 Alex Krizhevsky 是“非常强的 hacker”，用两块 GPU 训练了一个星期——在当时实验通常只跑几分钟
- Caffe “非常大幅度地推动了 computer vision 领域 adopt 深度学习”
- 深度学习框架的出现，让搭建模型从“6 个月写 2 万行 CUDA”变成“几分钟写几行 Python”

算法、数据、算力的三角关系

“现在大家都知道人工智能依赖于数据、算力和算法。但在我们刚开始做的时候，大部分的关注点都在算法上面。”系统的进步（GPU 加速、分布式训练、自动求导框架）才让算法研究者能快速迭代实验，这种使能关系常常被忽视。

8.3 学术界的机遇与挑战

在学术界做 ML 系统研究面临的资源差距是现实的——学校百卡集群 vs 前沿实验室的万卡级别。主持人直接追问：“百卡和万卡还是差两三个数量级。”但陈天奇认为这个差距并没有想象中那么致命：

- **资源约束催生创造力**：“Lack of resource is also an opportunity to make people more creative.”这就是为什么他仍然坚持做机器学习编译——因为在学校不可能靠堆 100 个人来解决问题。
- **开源社区合作**：学校不能是一个个体，需要做出能让开源社区共建的工作。像 VRL、Magiline 等成功的项目都有高校因子。
- **垂直突破**：不需要做所有东西，但可以做关键模块做到最好。例如他们做的 XGrammar (结构化数据生成) 已成为开源社区做 structured generation 的事实标准之一——agent 输出的 JSON 格式是否正确，这个问题“非常重要，但实际上现在的解决方案都做的不好”。
- **R&D 层面的差距没那么大**：“很多时候工业界也没有那么富，特别是如果你不是在真正做 pre-training 的团队。”CMU 最近成立的 Flame Center 有百卡级别的集群，已经足以做很多 kernel 优化和系统研究。

关于“做问题已经被做光了”的焦虑

陈天奇讲了一个故事：刚开始做机器学习时，戴文渊坐在他旁边听一个关于 SVM 优化的报告，感叹说“如果早两年入行就好了，这些 elegant 的问题似乎都被做光了”。那是 20 年前的事。“It turns out there is always new problem.”每个时代都有自己的新问题和新的挑战，重要的是能够发现并解决它们。

8.4 MLSYS 的未来方向

陈天奇一直是 MLSys Conference 背后的重要推动者。他认为机器学习系统“就像数据库系统一样，是一个问题驱动的方向”。未来几年的重要方向包括：

1. **大模型训练与推理优化**：如何更高效地做 pre-training、fine-tuning 和 serving
2. **硬件适配** (Machine Learning Compilation)：硬件的 bring-up 和软件适配仍然是巨大瓶颈
3. **AI for Systems**：用 AI Agent 来优化 ML System Engineering 本身
4. **Agent 系统基础设施**：结构化生成 (如 XGrammar)、tool calling 等

模型趋同会让 MLSYS 衰退吗？

主持人提出了一个尖锐的问题：大语言模型越来越趋同 (Top 5 模型可能占了 99% 的 inference 量)，这会不会让 MLSYS 失去存在价值？陈天奇承认这种可能性不能完全否认，但实际情况更复杂。一方面，硬件本身“has to be specialized”——每一代芯片的编程模式都不同。另一方面，“Hardware Lottery”意味着当前模型形态可能不是最优的——深度学习之所以繁荣，就是因为百花齐放。MLSYS 不仅是优化当前几种模型，更是 enable 新模型形态的探索。

8.5 本章小结

机器学习系统的演化始终与算法突破相伴相生。系统不是锦上添花，而是使能技术。学术界在这个领域仍有独特优势：中立性、长期投入的耐心、以及开源社区的天然契合度。陈天奇特别强调，虽然学术界在规模上无法与前沿实验室竞争，但“资源约束催生创造力”——做关键模块做到最好、通过开源社区合作扩大影响力，是学术界的独特路径。

9 OctoML：从开源到创业

9.1 创业动机

OctoML 的创立非常自然：TVM 项目有几个同学和老师，觉得有商业化机会，就决定做 startup。陈天奇的核心动机是“希望开源社区能够成功”——为了开源去做 startup，而不是为了 startup 去做开源。

他第一年全职参与，之后转为 part-time，主要负责技术方向和下一代技术研究。他“很明确不要当 CEO”，因为“你要放弃技术很多”，而他的 passion 在于做好的技术。

9.2 商业模式的转变

OctoML 的商业模式经历了显著转变：

1. **早期 (2019–2022)**：通过机器学习编译帮助硬件厂商和客户更快部署模型到各种硬件。潜在客户包括芯片厂商和需要部署 ML 的企业。
2. **后期 (2023–2024)**：转型为 LLM API 服务商（类似 Fireworks、Together），直接提供 Llama 等模型的 endpoint，靠 API 调用收费。机器学习编译技术退居幕后成为支撑技术，帮助降低 inference 成本。
3. **2024 年底**：被 NVIDIA 收购，团队负责 NVIDIA 的 AI compiler 方向。

主持人追问 API 市场竞争激烈，大家都在比谁的 Llama inference cost 更低，OctoML 如何应对？陈天奇承认竞争激烈，但指出产品形态上仍有差异化空间，比如支持 multi-LoRA inference 等。他也坦言，NVIDIA 收购并非预设的退出路径——“我完全没有想过这个问题，当时就觉得能够把一件事情做好看看就好。”

导师创业经历的预示

在加入 OctoML 之前，陈天奇已经目睹了导师 Carlos Guestrin 的创业公司 Turi（前身 GraphLab/Dato）的演变：从 CMU 的大规模图处理软件，到被收购时已完全转型为 Data Science 平台。“所以在开始做这件事情的时候，就会有心理准备——你去开始做的事情和 what you end up doing 一定会是不一样的。” Databricks 从 Spark 出发，现在 Spark 也不再是其最主要的部分。这是开源 startup 的普遍规律。

开源与商业的微妙关系

陈天奇坦承开源项目做 startup 会面临“开源太好用导致没人付费”的问题。但他的观点是：（1）开源是你的起点，但几乎一定不是终点——Databricks 现在的核心业务也不再是 Spark；（2）所有 startup 开始想做的东西和最后做的东西“一定是不一样的”；（3）既然选择了开源，就要接受它的好处（社区反馈、影响力）和挑战（商业模式设计）。

9.3 创业最大的 Lesson

创业五年给陈天奇带来了技术之外的重要认知：

- **Human factor is very important**: “做 startup 最大的 lesson 是——很多时候它不光有技术的因素。怎么样跟人交流，怎么样在大的团队中扮演一个 leadership 的 role，我觉得这一点非常重要。”
- **必须 reinvent yourself**: 不能抱着原来的技术路线不变。“如果我们两年前还在继续优化 ResNet，这个我觉得最后我们会输得很惨。”在快速变化的 AI 领域，技术转型的速度和勇气至关重要。
- **技术 excellence 始终没有错**: “这个观点不会被所有人认同。但在我看来，很多时候做优秀的技术始终是没有错的。因为 in the end, you have to have something to differentiate yourself。”
- **理想主义 + 现实主义**: “我有一点点理想主义。但这段经历会让我觉得我会更现实——我会知道说怎么样能够通过现实的方式去更好地实现理想。”
- **Product Market Fit**: 需要思考你的产品到底解决了谁的什么问题。但同时不能只看短期 PMF 而忽视技术的长期价值。
- **明确自己的角色**: “我很明确我不要当 CEO。因为当 CEO 很难很难，你要放弃技术很多。我的 passion 在于做好的技术。”了解自己擅长什么、热爱什么，然后找到合适的 co-founder 互补。

开源与商业的微妙关系

陈天奇坦承开源项目做 startup 会面临“开源太好用导致没人付费”的问题（主持人举了 Anyscale 和 Ray 的例子）。但他的观点是：（1）开源是你的起点，但几乎一定不是终点——Databricks 现在的核心业务也不再是 Spark；（2）所有 startup 开始想做的东西和最后做的东西“一定是不一样的”；（3）既然选择了开源，就要接受它的好处（社区反馈、影响力、用户信任）和挑战（商业模式需要差异化）。上一期嘉宾胡渊鸣的太极（Taichi）也遇到了类似问题——“很多人都说太极很好，但不愿意花钱。”这是基于开源创业的普遍挑战。

9.4 本章小结

OctoML 的五年历程展示了学术开源项目商业化的典型路径：从技术驱动到产品驱动，从编译工具到 API 服务，最终被战略收购。陈天奇在这个过程中学到了技术之外的重要维度：人的因素、产品市场匹配、以及持续自我革新的必要性。

10 开源的信仰与实践

10.1 为什么坚持开源

陈天奇是 open source、open science 的坚定信仰者。他认为开源的价值体现在多个层面：

1. **社区交流**：相互分享、认识不同的人、获得代码点赞的正反馈
2. **高校落地的路径**：高校本身不容易直接落地技术，开源是分享给大家、共建的最好方式
3. **学生培养**：在公司里你通常从大项目的小部件做起，但做开源项目的 PhD 有机会**从头架构一个系统**——这种训练极其珍贵
4. **高校的天然优势**：中立性 + 社区奉献精神

10.2 构建开源社区的关键

成功开源项目的三要素

1. **正确的技术**：让大家 believe in it。技术本身是基础，没有好的技术，社区不会聚集。
2. **持续维护的 passion**：一个软件不是以某个时刻的代码形态决定成功，而是需要被不断维护。Issue 和用户反馈“非常宝贵”——它们是迭代改进的最好输入。
3. **Build trust**：如果做了一个东西突然不维护了，大家在下一个项目发布时就会怀疑“他到底是不是需要相信这件事情”。信任需要长期积累，而且一旦失去很难重建。

陈天奇自己会“经常去看” issue 和用户反馈。他认为开源不是 free 的——需要大量时间看反馈、修 bug、跟社区互动，这些在传统科研评价体系中并没有直接回报（不算论文发表）。“所以并不是所有的组几乎应该是很少的组会采取这种路线，但它是一种方式。”

主持人追问：你所有开源项目的 issue 都会一个一个去看吗？陈天奇回答会经常去看，因为“一个软件它不可能以它某一个时候的代码形态来决定它的成功——它是要被不断维护的”。

10.3 本章小结

开源对陈天奇不仅是一种技术传播方式，更是一种价值观：让技术被更多人使用，让每个人都能创造自己的 AI 系统。这也是他做 OctoML 的初心——“为了开源社区能够繁荣”。

11 教职生涯：在一线写代码的教授

11.1 选择 CMU

陈天奇选择教职的三个原因：喜欢科研（享受不确定性）、喜欢和学生一起解决新问题、喜欢教大家做东西。他在 CMU 机器学习系任教，同时带博士生做 ML 系统方向的研究。他在快毕业时就已经决定走教职路线，而不是纯粹做工业界。

在被问到“如何说服最优秀的人来读博而不是去 XAI、OpenAI 搞 infra”时，他回答：“博士和其他路径不一样的地方是，it's like a longer investment——你会有足够多的时间和足够多的自由度去做一些事情。”学术界在 open science 的 alignment 上、在犯错的 tolerance 的时间尺度上，始终是更大的。他相信“还是会有有想法的同学愿意选择这条路”。

11.2 坚持一线写代码

与很多“从当教授第一天就再也没写过代码”的教授不同，陈天奇至今仍在一线做 git commit。最近一年他在从 ground up 重构 TVM 的核心部分，“foundation 的部分是我在做”。写代码对他来说是“recreational”的——更像是兴趣爱好。

AI 工具让教授写代码成为可能

陈天奇坦言：“如果没有 AI based tools，我做重构会困难很多。但 with the helpful tools (他用 Cursor), it's possible。”AI 编程工具降低了保持 hands-on 的门槛，让他即使在管理团队的同时也能保持对技术一线的感觉。他认为：“很多时候你需要在一线，可以 get a good sense of what's going on。”

11.3 捕鱼的比喻：积淀与时机

面对“AI research 节奏加快，以年为单位做项目还行不行”的问题，陈天奇用了一个比喻：

“做研究就像捕鱼。你可以拿叉子看着鱼游过去插它，但很多时候插不准。另一个方式是慢慢编一张网，当鱼来的时候，撒了几次之后或许可以抓到。”

他认为优秀的工作都需要积淀，很多时候看似“快速推出”的成果，其实是前面长期积累的结果。

11.4 Advising style

他的 advising style 延续了 Carlos 的传统并加入了自己的特色：

1. **做有影响力的工作**：这是从 Carlos “write the best paper on the wall” 传承下来的
2. **重视 Presentation**：延续 Carlos “no beamer, only PowerPoint” 的传统，强调论文和展示要让人能快速学会你做的东西
3. **鼓励团队合作**：在 ML 系统方向，把项目从 research 推动到真正的 adoption 需要团队协作
4. **培养 independent student**：他坚信学生应该有 drive things forward 的能力，自己则 act as a support role
5. **少做项目、深挖每一个**：“我们 deliberately 地希望能够少做一些东西，集中做我们觉得可以深挖的项目”——这也是为什么如果只看论文发表数量，他的产出并不算“高效”

他管理时间的方式是有所选择和 focus：知道最想要的是什么，然后去做。同时幸运地有非常 independent 的学生和 co-founder 分担日常运营。

11.5 教职生涯本章小结

陈天奇选择了一条少见的道路：教授 + 创业 + 一线写代码。他通过 focus（少做项目、深挖每一个）、delegation（幸运有 independent 的学生和 co-founder）、以及 AI 工具的帮助来管理时间。他用“捕鱼”的比喻强调积淀的重要性：与其拿叉子追热点，不如在自己的领域慢慢编一张网。

12 人生哲学：长期主义、初心与勇气

12.1 长期主义的数学基础

陈天奇在《机器学习科研这十年》中写道：“很多困难和无助只是随机涨落的一部分，只要付出足够多的时间和耐心，随机过程总会收敛到和付出相称的稳态。”

他说这句话“就是一个定理”——随机过程的 time average 确实会收敛到稳态分布。但知道定理是一回事，在挫折中坚持是另一回事。

接受失败的能力

“失败也没那么可怕，然后要接受可以失败这个事实。”这个信念从高中 OI 只拿二等奖、到本科做深度学习两年无果、到创业的各种转型，一直在被强化。陈天奇认为：**经历越多失败，就越敢放手做事**——因为“失败了也没什么大不了的，我还是活得好好的”。早期失败甚至是一种“保护”：如果一开始就太成功，反而可能形成路径依赖。

12.2 勇气来自过去的自己

陈天奇的一个深刻洞察是：**做事情的勇气并不随经验增长而自动增加**。恰恰相反，看到的东西越多，反而越容易畏首畏脚。

“我觉得很多时候你的勇气可能是你过去的自己给你自己的约定。你要达成过去自己对自己的约定，才要继续去坚持做这个 style 的工作。”

他把这个过程描述为一个循环：过去的自己给当下的自己勇气，当下的自己在做的过程中不断鼓励自己，再把这份勇气交给未来的自己。

12.3 保持初心是最难的事

当被问到“展望未来 5-10 年想干什么”时，陈天奇的回答出人意料：

“我觉得现在的我最难的事情，还是要保持初心。”

他解释说，随着资源变多、选择变多、责任变大，反而更难保持当年“出生牛犊不怕虎”的心态。做 MXNet 时完全不担心 TensorFlow 的竞争，但现在面对一个新项目时会不由自主地问“它到底能不能成功”。保持初心需要刻意的自我提醒。

主持人追问：你能调动的资源比 10 年前多了很多个数量级，为什么反而觉得保持初心更难？陈天奇回答：“拿到资源的同时必然代表着责任。你必须保证团队可以成功，必须产出相应的 impact 来 justify 做相关的事情。如果被这些压力裹挟而走，就会失去做 risky 事情的勇气。”

他的应对策略是：在 CMU 团队中继续做更 risky 的事情，“愿意放弃和愿意接受失败——这个心态其实和一开始是一样的”。资源和团队变大了，能做的事的确比原来更多了，但心态上的转变可能让人“忘记自己的那个心态”。

成功后的心态陷阱

这段讨论揭示了一个深刻的悖论：成功和经验的积累可能**侵蚀做创新所需的勇气**。看到的东西越多、知道的风险越多，反而越容易保守。陈天奇认为，对抗这种倾向的方法是回到初心——不是用理性的投入产出分析来做决策，而是问自己：“这是不是我理想中想要走的路？”如果是，就不管结果如何都去做。

12.4 关于成功与幸福

主持人在最后部分问了几个个人化的问题。

关于成功：陈天奇认为成功有很多定义——影响力、财富、技术突破等。但对他而言，最重要的是“做问心无愧的事情”。“能够在技术上面可以影响大家，并且可以为这个世界一起做一些贡献，就是一件好事情。”

关于对未来自己的寄语：如果让他给过去和未来的自己各说一句话，他的选择不是回头改变什么，而是相反——**他需要过去的自己来提醒现在和未来的自己：**记住对自己的承诺，坚持理想，往下走下去。

关于理想的人生状态：10 年后 20 年后理想的陈天奇，应该是“有更多次失败的一个陈天奇——并且可以继续保持那一个最初的心态去挑战我们想要挑战的东西”。

12.5 人生哲学本章小结

陈天奇的人生哲学可以用三个关键词概括：**长期主义**（相信随机过程会收敛）、**初心**（做问心无愧的事）、**勇气**（接受失败，不被资源和责任裹挟）。这些不是空洞的口号，而是在 20 年的科研和创业中被反复验证的行动准则。值得注意的是，他并不认为这些品质随着经验增长会自然增强——恰恰相反，他认为保持初心和勇气需要持续的、刻意的自我提醒。

13 总结与延伸

13.1 一条清晰的技术主线

回顾陈天奇近 20 年的职业生涯，一条主线贯穿始终：**让机器学习系统更高效、更易用、更可部署**。每一个项目都是对前一个项目瓶颈的回应：

- 做 KDD Cup 发现需要高效的树模型 → **XGBoost**
- 做 XGBoost 时重新燃起对深度学习的热情 → **CXXNet/MXNet**
- 做 MXNet 发现工程人力是瓶颈 → **TVM**（用编译技术替代手写 kernel）
- 做 TVM 需要在垂直领域验证 → **MLC LLM**（大模型端侧部署）
- TVM 的商业化需求 → **OctoML**（开源 → 创业 → 被 NVIDIA 收购）

项目	时期	解决的问题
XGBoost	2014	让树模型在大规模数据上跑得更快、更易用（自动处理 missing value）
MXNet	2015	让深度学习框架支持 Python first、自动调度、多卡并行、自动求导
TVM	2017	让 ML 模型自动适配各种硬件（GPU/TPU/NPU），减少手写 kernel
MLC LLM	2023	让大语言模型跑在手机、浏览器、MacBook 等端侧设备上

表 2: 陈天奇的技术贡献时间线

这条主线的底层逻辑是：**ML 系统不应该那么复杂，应该让更少的人通过正确的技术就能快速搭建**。这个信念从 XGBoost 时代（一个人做到极致）到 TVM 时代（编译器自动生成 kernel）到 MLC LLM 时代（端侧部署跨平台），一以贯之。

13.2 对年轻研究者的启示

综合访谈内容，陈天奇给年轻人的核心建议可以总结为以下几点，每一点都有他亲身经历的支撑：

1. **挑正确的问题**：“做平庸的问题和重要的问题花的时间几乎一样。”这是交大敖平老师教给他的，也是他 20 年来反复验证的。ImageNet 是正确的问题，RBM 是错误的方法——问题的选择比方法的选择更重要。
2. **不要同时锁死问题和方法**：这是他在交大做 ImageNet 两年无果后学到的“最宝贵的教训”。挑了问题后，方法要灵活调整——后来在 KDD Cup 中果断从 RBM 转向矩阵分解，就是这个教训的应用。
3. **做到极致**：Carlos 的要求是“write the best paper in the world”。不是追求论文数量，而是每一个项目都要做到 best paper level。XGBoost 的成功就是“一个算法做到极致”的最好例证。
4. **接受失败**：“做很难的问题必然会意味着不确定性和失败。”早期失败反而是好事——让你更敢放手做事，避免路径依赖。
5. **追随内心**：选择时间问自己“如果失败了会不会后悔”。这不是理性的投入产出分析，而是确保走的是理想中的路。
6. **从 first principle 出发**：不怕进入陌生领域。从写 transpiler 到做编译器到设计 NPU，陈天奇每次都是“出生牛犊不怕虎”地直接上手。系统设计能力可以在实践中磨练出来。
7. **开源是高校学生的独特机遇**：在公司里你从大项目的小部件做起；在开源项目中你有机会从头架构一个系统。这种训练在其他地方很难获得。

给 PhD 学生的特别建议

当被问到“如果你今天是三年级 PhD 学生，只能选择修复或改变一件事情，会做什么”时，陈天奇的回答是：“我应该会选择做我现在在做的事情。”也就是继续做机器学习编译——让更少的人用更少的 engineering resource 解决 ML 系统在最新硬件上的 bring up 问题。他不会因为时代不同就改变方向，因为这个问题“远远没有被解决”。

13.3 对 AI 未来的展望

陈天奇对 AGI 的定义不感兴趣（“我对 AGI 本身怎么定义并不是特别感兴趣”），更关心“pragmatically 有用”：能不能让一个人的团队撬动 100 人团队的产出？现在的技术“的确能够让我们更快地向那个目标前进”——比如他自己在用 Cursor 做 TVM 重构，如果没有 AI 工具“做重构会困难很多”。

对于 AI 世界的未来格局，他希望看到百花齐放——开源和个体都能做创新，而不是几家巨头主导一切。当被问到“那样的世界出现的概率是在增加还是减少”时，他的回答很谨慎：“我只能说它需要每一个人的努力。肯定会有不同人会有不同的信念。”

他坚持的技术路线始终是：**用更少的资源、更自动地做更多的事情**。“或许它在几年之内会被解决，或许可能不一定会被解决，但至少它是一个很有趣的问题。”

这句话也许是对陈天奇整个职业生涯最好的概括：不追求确定性的成功，而是追求有趣的问题。只要问题足够重要、足够有趣，就值得投入——即使结果是失败，也是“再出发的过程”。

13.4 结语

访谈的最后，主持人留了一段开放时间让陈天奇自由发挥。他说：

“在整个过程中我非常幸运能够获得众多人的帮助。最让我感觉收获深刻的是：我们还是可以去接受失败。在接受失败的过程中，继续鼓励自己做有趣的事。每个过程其实都是一个再出发的过程。We need to reinvent ourselves, then keep doing the next thing.”

如果让他给过去和未来的自己各说一句话，他的选择不是从现在回头教过去的自己什么，而是相反——他需要过去那个“出生牛犊不怕虎”的自己去提醒现在的自己：**记住对自己的承诺，坚持理想，往下走下去。**

主持人总结这段访谈说：理想的 10 年后 20 年后的陈天奇，应该是一个“有更多次失败的陈天奇——并且能够继续保持那一个最初的心态去挑战我们想要挑战的东西”。陈天奇欣然同意：“会有更多的失败，并且可以继续保持那个最初的心态去挑战。这样子最好了。”

访谈核心金句

以下是本次访谈中最具启发性的几段原话：

- “你去做一件比较平庸的问题和一个重要的问题，花的时间几乎是一样的。”
- “时间是有限的，所以一定要做想要做的事情。”
- “很多时候你的勇气可能是你过去的自己给你自己的约定。”
- “如果选了这条路并且失败了，你还会不会后悔？”
- “失败也没那么可怕。然后要接受可以失败这个事实。”
- “我觉得现在的我最难的事情，还是要保持初心。”
- “做优秀的技术始终是没有错的。”
- “We need to reinvent ourselves, then keep doing the next thing.”

13.5 拓展阅读

- 陈天奇，《机器学习科研的十年》，知乎，2019。这篇文章回顾了他从 2009 年到 2019 年的科研历程，其中很多故事在本次访谈中有更详细的展开。文章中“随机过程总会收敛到和付出相称的稳态”这句话被广泛传播。
- XGBoost 论文：Chen & Guestrin, “XGBoost: A Scalable Tree Boosting System”, KDD 2016。这篇论文至今引用近 7 万次，是 KDD 历史上引用最高的论文之一。
- MXNet 论文：Chen et al., “MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems”, NeurIPS Workshop 2015。作者按字母序排列，体现了 PhD 学生之间“纯粹的合作”精神。

- TVM 论文: Chen et al., “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning”, OSDI 2018。开创了机器学习编译这个研究方向。
- LIME 论文: Ribeiro, Singh, Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier”, KDD 2016。Carlos 实验室同年另一篇开创性工作, 开启了可解释机器学习方向。
- MLC LLM 项目: <https://github.com/mlc-ai/mlc-llm>。将大语言模型编译部署到各种端侧设备。
- Apache TVM 项目: <https://tvm.apache.org/>。机器学习编译的开源基础设施。
- MLC 课程 (陈天奇在 CMU 讲授的机器学习编译课程): <https://mlc.ai/>。系统讲解机器学习编译的原理和实践。
- XGrammar: <https://github.com/mlc-ai/xgrammar>。结构化数据生成的开源库, 已成为 LLM Agent 结构化输出的事实标准之一。
- WhynotTV Podcast 第二期 (胡渊鸣专访): 同样涉及开源项目商业化、学术创业等话题, 与本期形成有趣的对照。